

PROJECT REPORT

Title: RISC-V pipelined Data path

PAGES:

- Page 1 : Objective, Problem statement.
- Page 2 – 3 : Motivation for pipelining.(Why pipelining)
- Page 3 – 4 : Challenges for pipelining.
- Page 5 : Pictorial view of pipelined data path.
- Page 6 : Features of the data path. Role of instruction cache and program counter.
- Page 7 : Role of decoder unit, register file.
- Page 8 : Role Immediate generator, Data cache, 64-bit ALU.
- Page 9 : Control Signals
- Page 10 : Forwarding Unit
- Page 11-17 : Verilog code of data path and test bench.
- Page 17-24 : Observation of timing diagram and working of data path.
Timing reports
- Page 25 : Power consumption and further extensions possible

Designing Pipelined RISC-V Data path.

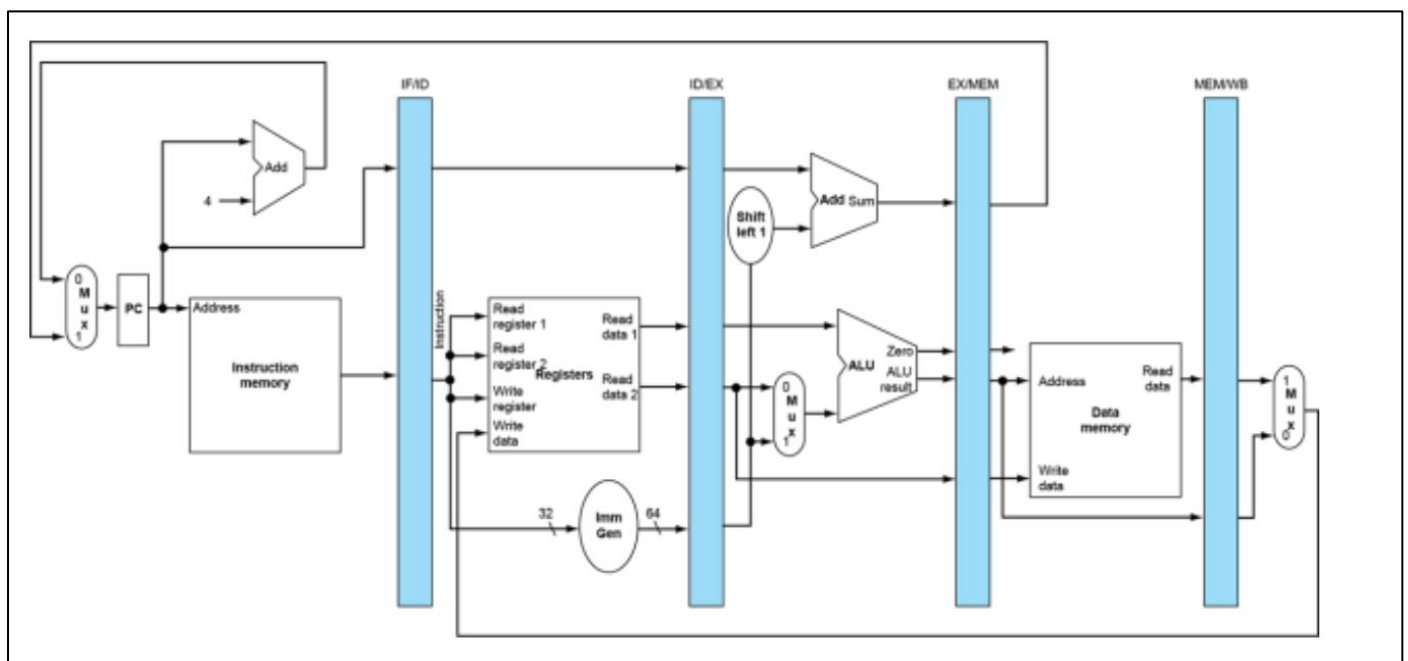
- **Objective:**

The objective of this LAB is to design a 4 stage RISC-V pipelined data path.

- **Problem Statement:**

Design a 5 stage RISC-V pipelined data path with the following features

1. Store data and access data.
2. Perform add, sub, mul, addi arithmetic operations.
3. Apply forwarding logic to decrease number of stalls as much as possible thereby increasing performance.



RISC-V 5 stage pipeline.

You can store value 45 and -20 in the D-Cache initially,

• Guiding Design principles in RISC-V ISA:

1. Simplicity favours regularity.
2. Smaller is faster.
3. Good design demands good compromises.

• Stages in RISC-V pipeline:

Similar to MIPS architecture there are 5 stages in RISC-V ISA (Instruction Set Architecture) :

- a) IF : Instruction fetch from Instruction cache.
- b) ID : Instruction Decode and Register read.
- c) EX: Execute Operations[eg: add, sub] (or) Calculate address[ld, sd].
- d) MEM: Access memory operands from data cache.
- e) WB: Write result back to the register in register file.

• Why pipelining?:

NOTE:

$$\text{Execution time} = \text{IC} * \text{CPI} * T$$

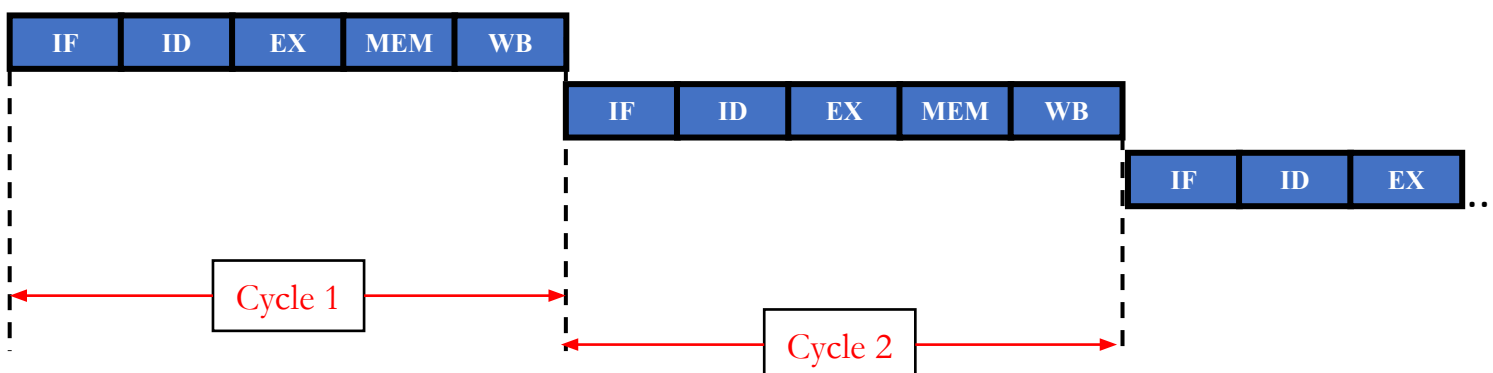
IC – Instruction count

CPI – Cycles Per Instruction

T – Clock Period.

In a single cycle processing unit the instructions are going to be executed sequentially one by one. Here CPI (Cycles Per Instruction) is low [for one instruction – one cycle] but here we are going to have a long clock period [the clock period corresponds to slowest instruction]. As a result the execution time is going to increase significantly due to long clock period, which results in less performance. This single cycle processing unit doesn't have any benefit except its simplicity.

Picture title: Diagram representing single cycle processing unit.

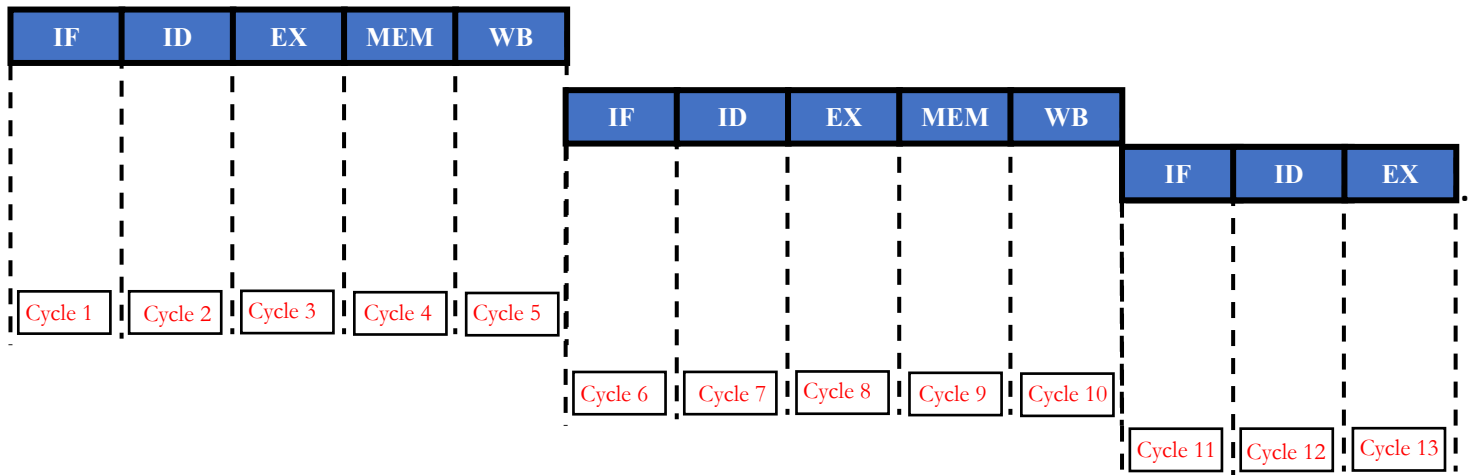


CPI: 1

Clock period: IF_time + ID_time + EX_time + MEM_time + WB_time
(long clock period, critical path: ld instruction)

In a multi-cycle processing unit each stage is executed in one cycle without pipelining. This results high CPI [one instruction 5 stages – 5 cycles] and short clock period [clock period corresponding to the longest time taken among 5 stages]. As a result execution time is going to increase and due to which performance decreases. Here one important thing to note that at each cycle, all hardware except the hardware corresponding to the stage executed in that cycle remains idle (unused).

Figure : Multicycle processing unit without pipelining.



CPI : 5

Clock period: $\max\{\text{IF_time}, \text{ID_time}, \text{EX_time}, \text{MEM_time}, \text{WB_time}\}$

• Challenges in pipelining:

- 1) Speed up issues when unbalanced.

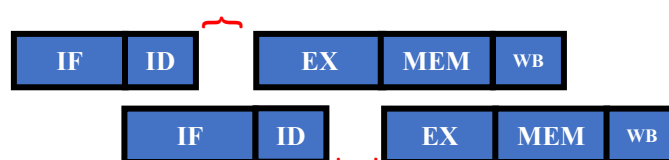
If the time taken by all stages is same (that is balanced) then we can say that

$$(\text{time})_{\text{pipelined}} = \frac{(\text{time})_{\text{non-pipelined}}}{\text{Number of stages}}$$

But in general the stages are not balanced, so we do not get the ideal speed up as written in the above formulae. The throughput will be less than expected. But the throughput of pipelined datapath will be far better than non-pipelined datapath.

Eg: if the time needed for ID stage is less than EX stage, when these both stages are executing in the same clock cycle, ID stage should wait for getting next computation until the EX stage gets completed.

Figure : ID , WB stages take less time(register operations) than other stages:



2) Hazards

a) Structural Hazards

Conflict for use of resource, IF and MEM stage requires memory or cache element but if there is only one cache then a requirement of bubble is happening, so this can be avoided by giving two different caches Instruction-cache and Data-cache.

b) Data Hazards

An instruction depends on completion of previous instruction, this type of issue is known as **Data dependencies**.

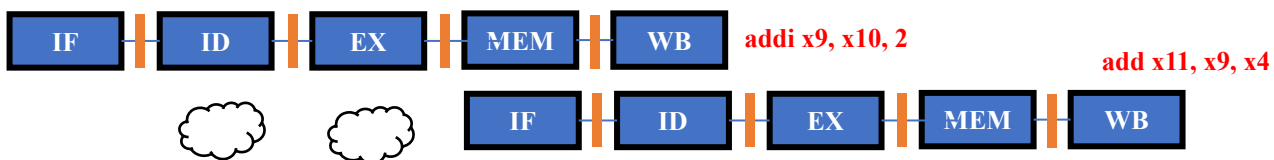
Eg:

1) I1: addi x9, x10, 2

I2: addi x11, x9, x4

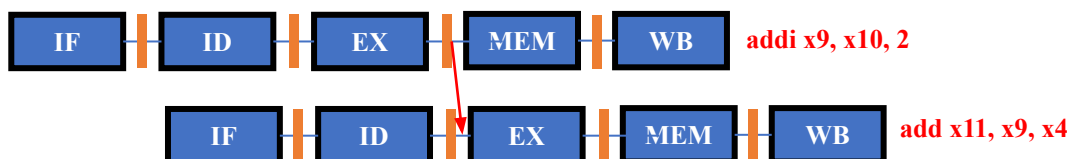
I2 depends on I1 because this is Write after read dependency

Figure : Bubbles needed (no forwarding) due to data dependency:



In the above case, WB is done in the first half of cycle and then ID.

Figure : Forwarding (aka bypassing) the above data dependency:

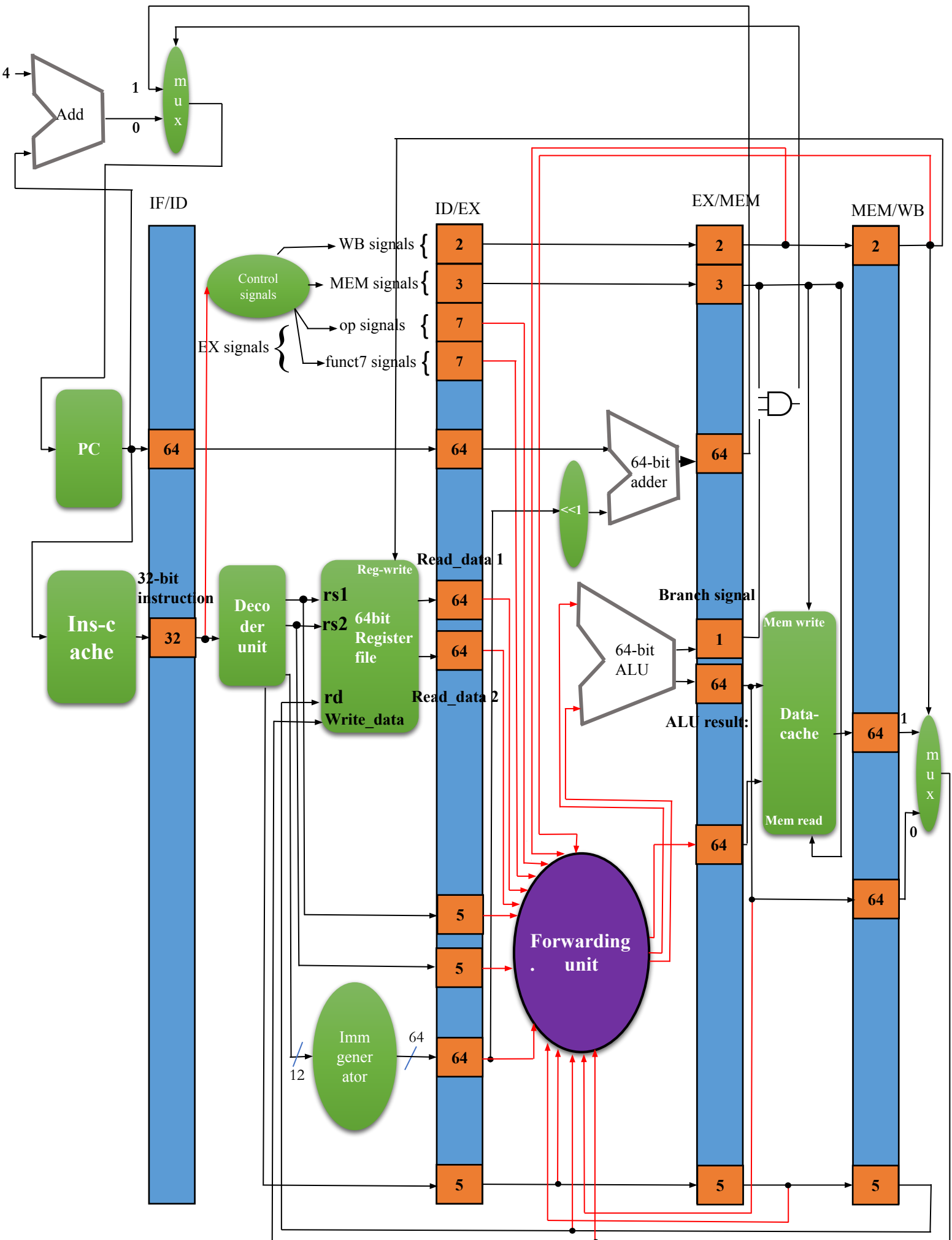


Not all data dependencies can be avoided by forwarding, if the instruction is ld instruction and next instruction is reading the register then there is a need of one bubble even when there is forwarding.

c) Control Hazards.

When a instruction is branch instruction the next instruction depends on the result of branch instruction. So there are many methods for overcoming this some of them are 1) Computing branch at the ID stage (this needs one bubble) 2) branch predictors (dynamic and static branch predictors). Handling control hazards is out of the scope of this project.

Figure : RISC-V pipelined data path



Note: Red wires depict the input and output of forwarding logic. The number in orange color boxes depicts the number of bits.

- **Goal:**

- 1) Use hardware lying as much as possible.(running instructions in parallel though there is no actual parallel hardware)
- 2) Minimize the number of stalls
- 3) Thereby increasing the performance.
- 4) Retaining the simplicity of the hardware.

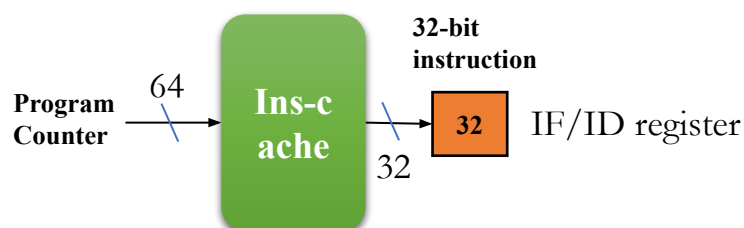
- **Features of this data path:**

Our main motive of this project is to come close as much as possible to the actual RISC-V pipelined data path. The following features are included in this data path:

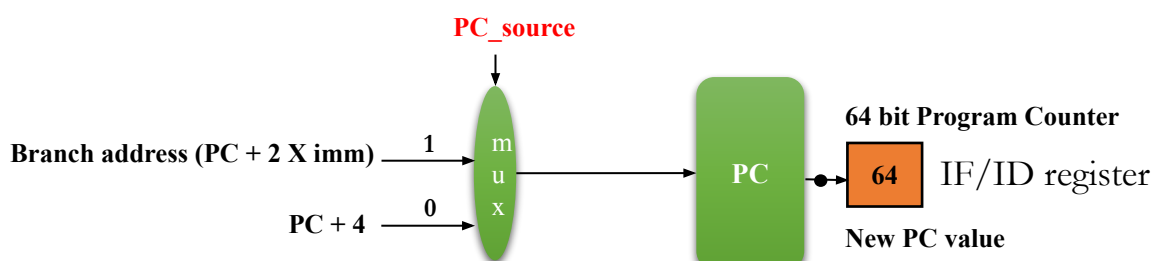
1. Pipelined data path: which allows multiple subtasks to be carried out simultaneously using independent resources.
2. Important components of Data path:
 - a) 20 32-bit instruction cache. (can be increased based on requirements)
 - b) Program Counter.
 - c) Instruction decoder. (R-type, I-type, S-type)
 - d) 32 64-bit register file.
 - e) Immediate Generator.
 - f) 64-bit ALU(addition, subtraction, multiplication)
 - g) 20 64-bit Data cache. (can be increase based on requirements)
 - h) Pipeline registers(IF/ID, ID/EX, EX/MEM, MEM/WB)
 - i) Control Unit. (Generate control signals)
 - j) Forwarding unit.

- **Description of Approach:**

Instruction cache stores instructions, when instruction is fetched from it using program counter.



Program counter is used for iterating through the instruction cache. Assuming the cache is byte addressable it is incremented by 4 to do to the next instruction and it is given $PC + 2 \times imm$ when there are branch instructions.



Role of decoder unit and register file in data path:

In this project the decoder unit decodes I, R, S type instructions of RISC-V. The register file has 32 64-bit registers where register read and register write gets executed.

32 64bit Registers in register file:

- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

R-format:

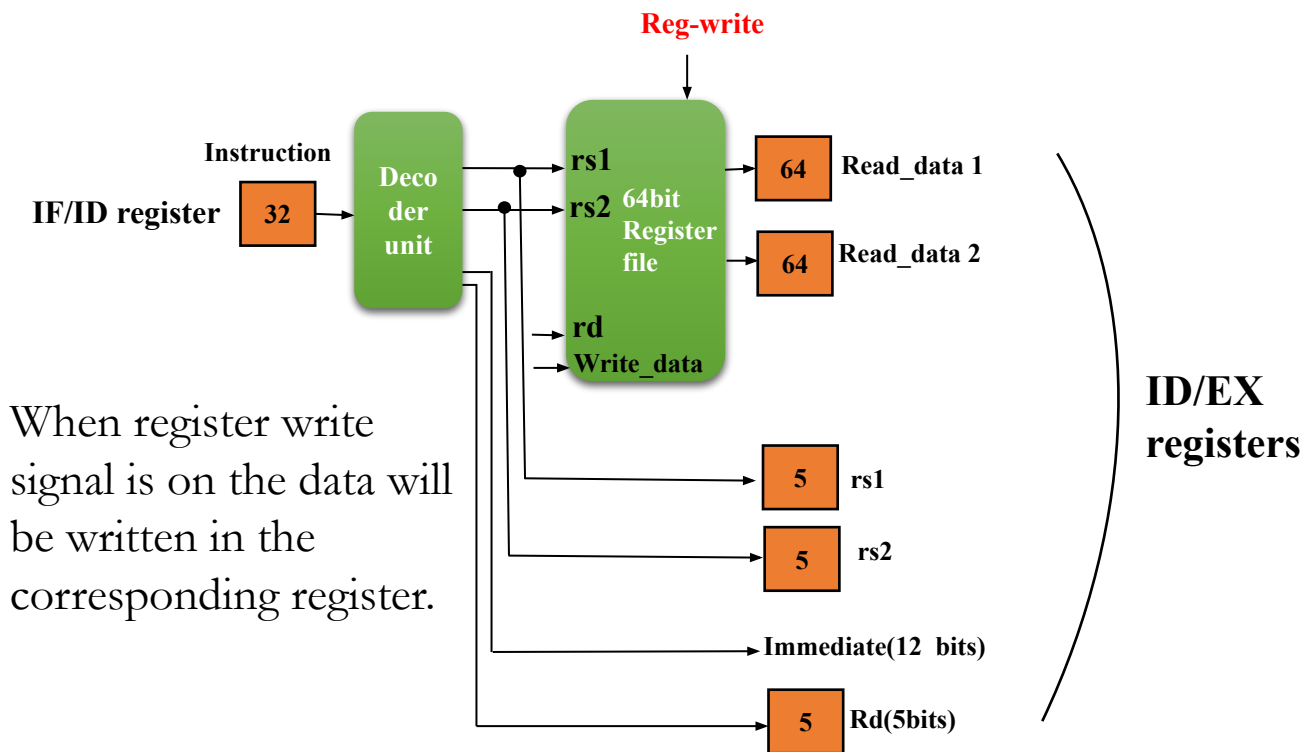
funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

I-format:

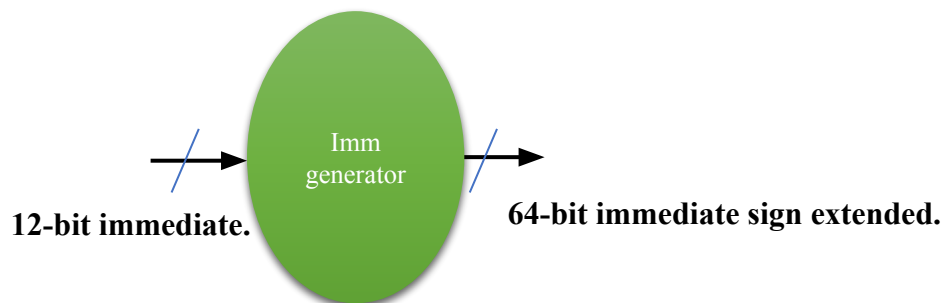
Immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

S-format:

Imm[11:5]	rs2	rs1	funct3	Imm[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

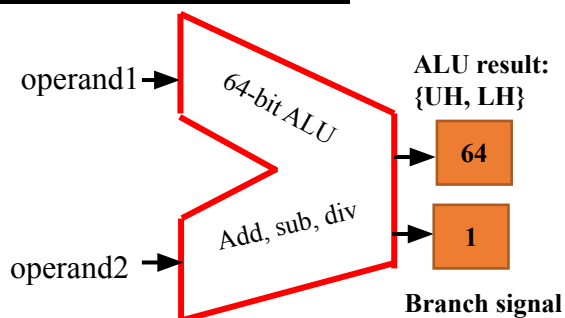


Role of immediate generator:



This imm generator performs sign extension of 12-bit immediate to 64-bit immediate. The reason for extension is that other registers in register file are 64-bit and the ALU is designed for 64-bits.

Role of 64-bit ALU:



In the data path designed here the ALU is designed for 64-bits and it performs addition and subtraction by carry look ahead adder logic and multiplication by booth's algorithm. It gives one output of 64-bits size.

Role of Data cache:

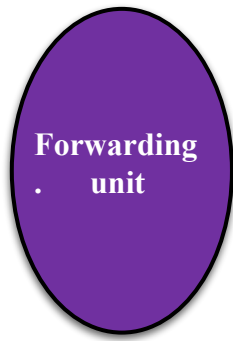
The data cache is used to store data values. The access time to access data from data cache is way more than access time to access data from register file. When data read signal is on data is read from data cache, whenever data write signal is on data is written in the data cache.

Control Unit and Control Signals:

S.No	Signals	Role
1)	Register write	If register write signal is 1 the data will be written in the register file. Otherwise data is not written in the register file.
2)	Mem_to_register	If mem_to_register signal is 1 the data read from the data cache (ld) will be write data value for register file. Otherwise alu_result will be the write data value for register file.
3)	Mem_read	If mem_read signal is 1, data value is read from the data cache using the address otherwise the data is not read from the data cache.
4)	Mem_write	If mem_write signal is 1, data value is written in the data cache else the data is not written in the data cache.
5)	PC_source	PC_source will be 1 when the instruction is branch instruction to select $PC + 2 \times \text{imm}$ Otherwise $PC + 4$ will be the updated program counter.
6)	ALU_source	This is execution signal, this depends on opcode and funct7 values. If instruction is R, based on funct7 value the type of operation like add, sub or mul is determined. And the operands will be data from rs1 and rs2. For other instructions like I format like addi, ld and S-type like sd only addition is needed to performed in execution stage, the addition involves immediate. One of the operand will be immediate.

Operation to be performed	Funct 7	Funct 3	Opcode
ADDITION(+)	1000000	000	0110011
SUBTRACTION(-)	0100000	000	0110011
MULTIPLICATION(X)	0000001	000	0110011

Working Of Forwarding Unit(aka by-passing):



The forwarding unit is represented by this block diagram but this forwarding unit can be viewed as several multiplexers placed together. The multiplexers involved in this forwarding logic can be seen from verilog code placed in page12 and page13 from this report.

Forwarding unit is added in this data path to decrease the number of stalls and thereby decreasing CPI and increasing performance of the processing unit.

The forwarding unit decides the operands to be sent to the ALU. The below conditions make sure that the most recent value is forwarded. The most recent value to forward will be there in EX/MEM pipeline.

1. If rs1 in ID/EX pipeline is equal to rd of EX/MEM pipeline, with write back in EX/MEM, and registers not being zero:
Then alu_result in EX/MEM is passed as operand A.
2. If the above condition is not satisfied and if rs1 in ID/EX is equal to rd of MEM/WB pipeline, with registers not being zero and write back is on.
 1. If write back signals in MEM/WB is indicating that the instruction in it was load then register value corresponding to the value read from data cache is passed.
 2. If not then alu_result from MEM/WB is passed.
3. If all the above conditions are not met then read_data1 from ID/EX is passed as operand A to ALU.

For operand B, the similar conditions are checked as above but additionally making sure that immediate is passed whenever instruction ID/EX is not R-type. As the conditions for passing B is complicated than A that can be verified from the Verilog code.

• Pipelined data path Verilog code:

```

1  module pipelined_datapath(
2      input clk,
3      input [31:0]instruction,
4      input instruction_memory_cycle,
5      input data_memory_cycle,
6      input [64:0]data,
7      input [3:0]data_location, // 10 locations needed
8
9
10     output [63:0]PC,
11     output [95:0]pipeline_IF_ID,
12     output [289:0]pipeline_ID_EX,
13     output [202:0]pipeline_EX_MEM,
14     output [134:0]pipeline_MEM_WB,
15     output reg [63:0]data_written_in_regfile,
16     output reg [63:0]data_written_in_data_mem,
17     output reg [31:0]data_written_in_ins_mem,
18     output reg overflow
19 );
20
21 /*
22 20 32-bit instructions can be stored in instruction memory
23 */
24 reg [31:0]instruction_memory[19:0];
25
26 /*
27 20 64-bit data can be stored in data memory
28 */
29 reg [63:0]data_memory[19:0];
30
31 /*
32 32 64-bit register file
33 */
34 reg [31:0]register_file[63:0];
35
36 /*
37 IF/ID pipeline
38 PC - 64 bits
39 instruction - 32 bits
40 total bits = 96
41 */
42 reg [95:0]pipeline_IF_ID;
43
44 /*
45 ID/EX pipeline
46 write back signals - 2 bits (register write signal , mem to reg signal)
47 memory signals - 3 bits
48             (memory read, memory write, branch)
49 execution : opcode - 7 bits
50             funct7 - 7 bits
51 PC - 64 bits
52 read_data1 = 64 bits
53 read_data2 = 64 bits
54 rs1 = 5 bits
55 rs2 = 5 bits
56 immediate sign extended = 64 bits
57 rd = 5 bits
58 total = 2 + 3 + 7 + 7 + 64 + 64 + 64 + 5 + 5 + 64 + 5 = 290 bits
59 */
60 reg [289:0]pipeline_ID_EX;
61
62 /*
63 EX/MEM pipeline
64 write back signals - 2 bits (register write signal , mem to reg signal)
65 memory signals - 3 bits
66             (memory read, memory write, branch)
67 new branch = 64 bits
68 alu_branch_signal = 1 bit
69 alu_result = 64 bits
70 read_data2 = 64 bits
71 rd = 5 bits
72
73 total_bits = 203 bits
74 */
75 reg [202:0]pipeline_EX_MEM;
76
77 /*
78 MEM/WB pipeline
79 write back signals - 2 bits (register write signal , mem to reg signal)
80 mem data = 64 bits
81 alu_result = 64 bits
82 rd = 5 bits
83
84 total bits = 135 bits
85 */
86 reg [134:0]pipeline_MEM_WB;
87
88 integer iterator = 0;
89 integer i;
90
91 reg [63:0]PC;
92
93 reg [63:0]A;
94 reg [63:0]B;
95 reg [63:0]B1;
96 reg [63:0]G;
97 reg [63:0]P;
98 reg [63:0]sum;
99 reg [64:0]carry;
100 reg [127:0]mul;
101 reg [1:0]temp;
102 reg EI;
103 reg [63:0]alu_result;
104
105 reg PC_source;
106
107 always @ (posedge clk)
108 begin
109     register_file[0] <= 0;
110     if (instruction_memory_cycle == 1'b1)

```

Instruction cache.

Data cache.

Register file.

Pipeline IF/ID

Pipeline ID/EX.

Pipeline EX/MEM.

Pipeline MEM/WB.

```

110 begin
111     // initially to put instructions in instructiojn memory
112     PC <= 0;
113     for(i = 0; i <= 31; i = i + 1)
114     begin
115         register_file[i] = 0;
116     end
117
118     pipeline_IF_ID <= 0;
119     pipeline_ID_EX <= 0;
120     pipeline_EX_MEM <= 0;
121     pipeline_MEM_WB <= 0;
122     alu_result <= 0;
123
124     instruction_memory[iterator] <= instruction;
125     iterator <= iterator + 1;
126
127     data_written_in_regfile <= 0;
128     data_written_in_data_mem <= 0;
129     data_written_in_ins_mem <= instruction;
130 end
131 else if (data_memory_cycle == 1'b1)
132 begin
133     // initially to put some data into the data memory
134     pipeline_IF_ID <= 0;
135     pipeline_ID_EX <= 0;
136     pipeline_EX_MEM <= 0;
137     pipeline_MEM_WB <= 0;
138     alu_result <= 0;
139     data_memory[data_location] <= data;
140
141     data_written_in_regfile <= 0;
142     data_written_in_data_mem <= data;
143     data_written_in_ins_mem <= 0;
144 end
145 else
146 begin
147
148 // =====
149 // instruction fetch
150 pipeline_IF_ID[63:0] <= PC;
151 pipeline_IF_ID[95:64] <= instruction_memory[PC >> 2];
152 // =====
153 // Write back stage
154 if (pipeline_MEM_WB[0] == 1'b1) // register write signal
155 begin
156     if(pipeline_MEM_WB[1] == 1'b1) // writing data memory result
157     begin
158         register_file[pipeline_MEM_WB[134:130]] <= pipeline_MEM_WB[65:2];
159         data_written_in_regfile <= pipeline_MEM_WB[65:2];
160     end
161     else
162     begin
163         register_file[pipeline_MEM_WB[134:130]] <= pipeline_MEM_WB[129:66];
164         data_written_in_regfile <= pipeline_MEM_WB[129:66];
165     end
166 end
167 else
168 begin
169     data_written_in_regfile <= 0;
170 end
171 // =====
172 // program counter calculation and memory stage
173 PC_source <= pipeline_EX_MEM[2] & pipeline_EX_MEM[69];
174
175 if (PC_source == 1'b1)
176 begin
177     PC <= pipeline_EX_MEM[68:5];
178 end
179 else
180 begin
181     PC <= PC + 4;
182 end
183
184 if(pipeline_EX_MEM[4] == 1'b1) // read(ld)
185 begin
186     pipeline_MEM_WB[65:2] <= data_memory[pipeline_EX_MEM[133:70]];
187
188     data_written_in_data_mem = 0;
189 end
190 else if(pipeline_EX_MEM[3] == 1'b1) // write(sd)
191 begin
192     pipeline_MEM_WB[65:2] <= 0;
193     data_memory[pipeline_EX_MEM[133:70]] <= pipeline_EX_MEM[197:134]; // read data 2
194
195     data_written_in_data_mem <= pipeline_EX_MEM[197:134];
196 end
197 else
198 begin
199     pipeline_MEM_WB[65:2] <= 0;
200
201     data_written_in_data_mem = 0;
202 end
203
204 pipeline_MEM_WB[1:0] <= pipeline_EX_MEM[1:0]; // write back signals
205 pipeline_MEM_WB[129:66] <= pipeline_EX_MEM[133:70]; // alu result
206 pipeline_MEM_WB[134:130] <= pipeline_EX_MEM[202:198]; // destination register
207 // =====
208 // Execution stage
209
210 // Forwarding Logic with multiplexers
211 if((pipeline_EX_MEM[202:198] != 5'b00000) && (pipeline_EX_MEM[202:198] == pipeline_ID_EX[215:211]) && (pipeline_EX_MEM[0] == 1'b1) && (pipeline_EX_MEM[1] == 1'b0))
212 begin
213     A = pipeline_EX_MEM[133:70]; // alu result
214 end
215 else if((pipeline_MEM_WB[134:130] != 5'b00000) && (pipeline_MEM_WB[134:130] == pipeline_ID_EX[215:211]) && (pipeline_MEM_WB[0] == 1'b1))
216 begin
217     if(pipeline_MEM_WB[1] == 1'b1)
218     begin
219         A = pipeline_MEM_WB[65:2]; // data from data_memory
220     end
221     else
222     begin

```

Initially writing some instructions into instruction-cache.

Initially writing some data values into data cache.

Instruction fetch stage.

Write back stage.

Program counter and MEM stage.

Forwarding unit

```

223     A = pipeline_MEM_WB[129:66]; // alu result
224 end
225 end
226 else
227 begin
228     A = pipeline_ID_EX[146:83]; // read data 1
229 end
230
231
232 if ((pipeline_EX_MEM[202:198] != 5'b000000) && (pipeline_EX_MEM[202:198] == pipeline_ID_EX[220:216]) && (pipeline_EX_MEM[0] == 1'b1) && (pipeline_EX_MEM[1] == 1'b0) )
233 begin
234     if (pipeline_ID_EX[11:5] != 7'b0110011)
235     begin
236         pipeline_EX_MEM[197:134] <= pipeline_EX_MEM[133:70]; // alu result
237         B = pipeline_ID_EX[284:221]; // immediate
238     end
239     else
240     begin
241         pipeline_EX_MEM[197:134] <= pipeline_ID_EX[210:147]; // read data 2
242         B = pipeline_EX_MEM[133:70]; // alu result
243     end
244 end
245 else if((pipeline_MEM_WB[134:130] != 5'b000000) && (pipeline_MEM_WB[134:130] == pipeline_ID_EX[220:216]) && (pipeline_MEM_WB[0] == 1'b1))
246 begin
247     if(pipeline_MEM_WB[1] == 1'b1) // ld
248     begin
249         if (pipeline_ID_EX[11:5] != 7'b0110011)
250         begin
251             pipeline_EX_MEM[197:134] <= pipeline_MEM_WB[65:2]; // data from data mem
252             B = pipeline_ID_EX[284:221]; // immediate
253         end
254         else
255         begin
256             pipeline_EX_MEM[197:134] <= pipeline_ID_EX[210:147]; // read data 2
257             B = pipeline_MEM_WB[65:2]; // data from data memory
258         end
259     end
260     else
261     begin
262         if (pipeline_ID_EX[11:5] != 7'b0110011)
263         begin
264             pipeline_EX_MEM[197:134] <= pipeline_MEM_WB[129:66]; // data from data mem
265             B = pipeline_ID_EX[284:221]; // immediate
266         end
267         else
268         begin
269             pipeline_EX_MEM[197:134] <= pipeline_ID_EX[210:147]; // read data 2
270             B = pipeline_MEM_WB[129:66]; // data from data memory
271         end
272     end
273 end
274 else
275 begin
276     if (pipeline_ID_EX[11:5] == 7'b0110011) // opcode (R-type instruction)
277     begin
278         B = pipeline_ID_EX[210:147]; // read data 2
279         pipeline_EX_MEM[197:134] = pipeline_ID_EX[210:147]; // read data 2
280     end
281     else // (S-type, I -type)
282     begin
283         B = pipeline_ID_EX[284:221]; // immediate
284         pipeline_EX_MEM[197:134] = pipeline_ID_EX[210:147]; // read data 2
285     end
286 end
287
288 // Performing operation on A and B
289 if(pipeline_ID_EX[11:5] == 7'b0110011) // opcode
290 begin
291     // R-format
292     if (pipeline_ID_EX[18:12] == 7'b1000000) // funct 7 values
293     begin
294         // addition
295         carry[0] = 0;
296         for(i = 0; i <= 63; i = i + 1)
297         begin
298             G[i] = A[i] & B[i];
299             P[i] = A[i] ^ B[i];
300             sum[i] = carry[i] ^ P[i];
301             carry[i + 1] = G[i] | (carry[i] & P[i]);
302         end
303         alu_result = sum;
304         if (A[63] == 0 && B[63] == 0 && sum[63] == 1) // adding two positive numbers giving a negative number is not possible
305         begin
306             overflow = 1'b1;
307         end
308         else if (A[63] == 1 && B[63] == 1 && sum[63] == 0) // adding two negative numbers giving a positive numbers is not possible
309         begin
310             overflow = 1'b1;
311         end
312         else
313         begin
314             overflow = 1'b0;
315         end
316     end
317     else if(pipeline_ID_EX[18:12] == 7'b0100000)
318     begin
319         // subtraction
320         B1 = ~(B) + 1;
321         carry[0] = 0;
322         for(i = 0; i <= 63; i = i + 1)
323         begin
324             G[i] = A[i] & B1[i];
325             P[i] = A[i] ^ B1[i];
326             sum[i] = carry[i] ^ P[i];
327             carry[i + 1] = G[i] | (carry[i] & P[i]);
328         end
329         alu_result = sum;
330         if (A[63] == 0 && B1[63] == 0 && sum[63] == 1) // adding two positive numbers giving a negative number is not possible
331         begin
332             overflow = 1'b1;
333         end
334         else if (A[63] == 1 && B1[63] == 1 && sum[63] == 0) // adding two negative numbers giving a positive numbers is not possible

```

Execution stage.

```

335 begin
336     overflow = 1'b1;
337 end
338 else
339     begin
340         overflow = 1'b0;
341     end
342 end
343 else if(pipeline_ID_EX[18:12] == 7'b0000001)
344     begin
345         // multiplication
346         B1 = ~(B) + 1;
347         mul = 0;
348         E1 = 0;
349         for (i = 0; i <= 63; i = i + 1)
350             begin
351                 temp = (A[i], E1);
352                 case (temp)
353                     2'd2 : mul[127:64] = mul[127:64] + B1;
354                     2'd1 : mul[127:64] = mul[127:64] + B;
355                 endcase
356                 mul = mul >> 1;
357                 mul[127] = mul[126];
358                 E1 = A[i];
359             end
360         if (B == 64'd9223372036854775808)
361             begin
362                 mul = -mul;
363             end
364         overflow = 1'b0;
365         alu_result = mul[63:0];
366     end
367 end
368 else if(pipeline_ID_EX[11:5] == 7'b0010011 || pipeline_ID_EX[11:5] == 7'b0000011 || pipeline_ID_EX[11:5] == 7'b0100011)
369     begin
370         // I-format or S-format
371         // addition
372         carry[0] = 0;
373         for(i = 0; i <= 63; i = i + 1)
374             begin
375                 G[i] = A[i] & B[i];
376                 P[i] = A[i] ^ B[i];
377                 sum[i] = carry[i] ^ P[i];
378                 carry[i + 1] = G[i] | (carry[i] & P[i]);
379             end
380         alu_result = sum;
381         if (A[63] == 0 && A[63] == 0 && sum[63] == 1) // adding two positive numbers giving a negative number is not possible
382             begin
383                 overflow = 1'b1;
384             end
385         else if (A[63] == 1 && B[63] == 1 && sum[63] == 0) // adding two negative numbers giving a positive number is not possible
386             begin
387                 overflow = 1'b1;
388             end
389         else
390             begin
391                 overflow = 1'b0;
392             end
393     end
394 else
395     begin
396         alu_result = 0;
397         overflow = 0;
398     end
399 end
400 // writing in the registers
401 pipeline_EX_MEM[1:0] <= pipeline_ID_EX[1:0]; // write back signals
402 pipeline_EX_MEM[4:2] <= pipeline_ID_EX[4:2]; // memory signals
403 pipeline_EX_MEM[68:5] <= pipeline_ID_EX[82:19] + (pipeline_ID_EX[284:221] * 2); // branch calculation
404 pipeline_EX_MEM[69] <= 0; // branch signal
405 pipeline_EX_MEM[133:70] <= alu_result; // alu result
406 pipeline_EX_MEM[202:198] <= pipeline_ID_EX[289:285]; // destination register
407 // =====
408 // INSTRUCTION DECODE
409 if( pipeline_IF_ID[70:64] == 7'b0110011)
410     begin
411         // R type instruction
412         pipeline_ID_EX[1:0] <= 2'b01; // write back signals
413         pipeline_ID_EX[4:2] <= 3'b000; // memory read, memory write, branch signal
414         pipeline_ID_EX[11:5] <= pipeline_IF_ID[70:64]; // opcode values
415         pipeline_ID_EX[18:12] <= pipeline_IF_ID[95:89]; // funct 7 values
416         pipeline_ID_EX[82:19] <= pipeline_IF_ID[63:0]; // PC value
417         pipeline_ID_EX[146:83] <= register_file[pipeline_IF_ID[83:79]]; // read_data 1
418         pipeline_ID_EX[210:147] <= register_file[pipeline_IF_ID[88:84]]; // read_data 2
419         pipeline_ID_EX[215:211] <= pipeline_IF_ID[83:79]; // rs1
420         pipeline_ID_EX[220:216] <= pipeline_IF_ID[88:84]; // rs2
421         pipeline_ID_EX[284:221] <= 0; // immediate
422         pipeline_ID_EX[289:285] <= pipeline_IF_ID[75:71]; // destination register
423     end
424 else if(pipeline_IF_ID[70:64] == 7'b0010011)
425     begin
426         // addi
427         pipeline_ID_EX[1:0] <= 2'b01; // write back signal
428         pipeline_ID_EX[4:2] <= 3'b000; // memory read, memory write, branch signal
429         pipeline_ID_EX[11:5] <= pipeline_IF_ID[70:64]; // opcode values
430         pipeline_ID_EX[18:12] <= 0; // funct 7 values
431         pipeline_ID_EX[82:19] <= pipeline_IF_ID[63:0]; // PC value
432         pipeline_ID_EX[146:83] <= register_file[pipeline_IF_ID[83:79]]; // read_data 1
433         pipeline_ID_EX[210:147] <= 0; // read_data 2
434         pipeline_ID_EX[215:211] <= pipeline_IF_ID[83:79]; // rs1
435         pipeline_ID_EX[220:216] <= 0; // rs2
436         // sign extending 12 bit immediate
437         pipeline_ID_EX[232:221] <= pipeline_IF_ID[95:84];
438         for(i = 233; i <= 284; i = i + 1)
439             begin
440                 pipeline_ID_EX[i] = pipeline_IF_ID[95];
441             end
442         pipeline_ID_EX[289:285] = pipeline_IF_ID[75:71]; // destination register
443     end
444 else if(pipeline_IF_ID[70:64] == 7'b0000011)
445     begin
446         // ld type instruction
447         pipeline_ID_EX[1:0] <= 2'b11; // write back signal
448         pipeline_ID_EX[4:2] <= 3'b100; // memory read, memory write, branch signal

```

Instruction decode stage.

```

449 ○ pipeline_ID_EX[11:5] <= pipeline_IF_ID[70:64]; // opcode values
450 ○ pipeline_ID_EX[18:12] <= 0; // funct 7 values
451 ○ pipeline_ID_EX[82:19] <= pipeline_IF_ID[63:0]; // PC value
452 ○ pipeline_ID_EX[146:83] <= register_file[pipeline_IF_ID[83:79]]; // read_data 1
453 ○ pipeline_ID_EX[210:147] <= 0; // read_data 2
454 ○ pipeline_ID_EX[215:211] <= pipeline_IF_ID[83:79]; // rs1
455 ○ pipeline_ID_EX[220:216] <= 0; // rs2
456 ○ // immediate sign extension
457 ○ pipeline_ID_EX[232:221] <= pipeline_IF_ID[95:84];
458 ○ for(i = 233; i <= 284; i = i + 1)
459 ○ begin
460 ○     pipeline_ID_EX[i] = pipeline_IF_ID[95];
461 ○ end
462 ○ pipeline_ID_EX[289:285] = pipeline_IF_ID[75:71]; // destination register
463 ○ end
464 ○ else if(pipeline_IF_ID[70:64] == 7'b0100011)
465 ○ begin
466 ○     // S type instruction
467 ○     pipeline_ID_EX[1:0] <= 2'b00; // write back signals
468 ○     pipeline_ID_EX[4:2] <= 3'b010; // memory read, memory write, branch signal
469 ○     pipeline_ID_EX[11:5] <= pipeline_IF_ID[70:64]; // opcode values
470 ○     pipeline_ID_EX[18:12] <= 0; // funct 7 values
471 ○     pipeline_ID_EX[82:19] <= pipeline_IF_ID[63:0]; // PC value
472 ○     pipeline_ID_EX[146:83] <= register_file[pipeline_IF_ID[83:79]]; // read_data 1
473 ○     pipeline_ID_EX[210:147] <= register_file[pipeline_IF_ID[88:84]]; // read_data 2
474 ○     pipeline_ID_EX[215:211] <= pipeline_IF_ID[83:79]; // rs1
475 ○     pipeline_ID_EX[220:216] <= pipeline_IF_ID[88:84]; // rs2
476 ○     // sign extending immediate
477 ○     pipeline_ID_EX[225:221] <= pipeline_IF_ID[75:71];
478 ○     pipeline_ID_EX[232:226] <= pipeline_IF_ID[95:89];
479 ○     for(i = 233; i <= 284; i = i + 1)
480 ○     begin
481 ○         pipeline_ID_EX[i] = pipeline_IF_ID[95];
482 ○     end
483 ○     pipeline_ID_EX[289:285] = 0; // destination register
484 ○ end
485 ○ else
486 ○ begin
487 ○     pipeline_ID_EX = 0;
488 ○ end
489 ○ end
490 ○ end
491 ○ endmodule

```

TEST Bench:

The below test bench is written to simulate different test cases.

```

1  /*
2  R-type instructions
3  add      1000000_rs2_rs1_000_rd_0110011
4  sub      0100000_rs2_rs1_000_rd_0110011
5  mul      0000001_rs2_rs1_000_rd_0110011
6
7  I-type instructions
8  addi     imm(12bits)_rs1_000_rd_0010011
9  ld       imm(12bits)_rs1_011_rs_0000011
10
11 S-type instructions
12 sd       imm[11:5]_rs2_rs1_011_imm[4:0]_0100011
13 */
14 `timescale 1ns / 1ps
15
16 module test_bench;
17     reg clk;
18     reg [31:0] instruction;
19     reg instruction_memory_cycle;
20     reg data_memory_cycle;
21     reg [64:0] data;
22     reg [3:0] data_location;
23
24     wire [63:0] PC;
25     wire [95:0] pipeline_IF_ID;
26     wire [289:0] pipeline_ID_EX;
27     wire [202:0] pipeline_EX_MEM;
28     wire [134:0] pipeline_MEM_WB;
29     wire [63:0] data_written_in_regfile;
30     wire [63:0] data_written_in_data_mem;
31     wire [31:0] data_written_in_ins_mem;
32     wire overflow;
33
34     pipelined_datapath uut(
35         clk,
36         instruction,
37         instruction_memory_cycle,
38         data_memory_cycle,
39         data,
40         data_location,
41
42         PC,
43         pipeline_IF_ID,
44         pipeline_ID_EX,
45         pipeline_EX_MEM,
46         pipeline_MEM_WB,
47         data_written_in_regfile,
48         data_written_in_data_mem,
49         data_written_in_ins_mem,
50

```

```

51 overflow
52 );
53
54 initial
55 begin
56     clk = 1'b0;
57
58     instruction = 32'b00000000000011101000111010010011; // addi x29, x29, 0 (I - type instruction)
59     instruction_memory_cycle = 1'b1;
60     data_memory_cycle = 1'b0;
61     data = 0;
62     data_location = 0;
63
64     #2
65     instruction = 32'b00000000000011110000111100010011; // addi x30, x30, 1 (I - type instruction)
66     instruction_memory_cycle = 1'b1;
67     data_memory_cycle = 1'b0;
68     data = 0;
69     data_location = 0;
70
71     #2
72     instruction = 32'b00000000000011101011111000000011; // ld x28, 0(x29)
73     instruction_memory_cycle = 1'b1;
74     data_memory_cycle = 1'b0;
75     data = 0;
76     data_location = 0;
77
78     #2
79     instruction = 32'b00000000000011110011110110000011; // ld x27 0(x30)
80     instruction_memory_cycle = 1'b1;
81     data_memory_cycle = 1'b0;
82     data = 0;
83     data_location = 0;
84
85     #2
86     instruction = 32'b10000001110111100000010100110011; // add x10, x28, x29
87     instruction_memory_cycle = 1'b1;
88     data_memory_cycle = 1'b0;
89     data = 0;
90     data_location = 0;
91
92     #2 |
93     instruction = 32'b00000000101011101011000000100011; // sd x10, 0(x29)
94     instruction_memory_cycle = 1'b1;
95     data_memory_cycle = 1'b0;
96     data = 0;
97     data_location = 0;
98
99     #2
100    instruction = 32'b00000000000011101011010110000011; // ld x11 0(x29)
101    instruction_memory_cycle = 1'b1;
102    data_memory_cycle = 1'b0;
103    data = 0;
104    data_location = 0;
105
106    #2
107    instruction = 32'b10000000000000000000000000110011; // add x0, x0, x0 (R-type instruction) STALL
108    instruction_memory_cycle = 1'b1;
109    data_memory_cycle = 1'b0;
110    data = 0;
111    data_location = 0;
112
113    #2
114    instruction = 32'b0000001010111110000001100110011; // mul x6, x11, x31 (R-type instruction)
115    instruction_memory_cycle = 1'b1;
116    data_memory_cycle = 1'b0;
117    data = 0;
118    data_location = 0;
119
120    // Putting -20 from data to register x18
121    #2
122    instruction = 32'b00000000000011110011100100000011; // ld x18 1(x30)
123    instruction_memory_cycle = 1'b1;
124    data_memory_cycle = 1'b0;
125    data = 0;
126    data_location = 0;
127
128    #2
129    instruction = 32'b10000000000000000000000000110011; // add x0, x0, x0 (R-type instruction) STALL
130    instruction_memory_cycle = 1'b1;
131    data_memory_cycle = 1'b0;
132    data = 0;
133    data_location = 0;
134
135    #2
136    instruction = 32'b01000001001011011000110110110011; // sub x19, x27, x18 (R - type instruction)
137    instruction_memory_cycle = 1'b1;
138    data_memory_cycle = 1'b0;
139    data = 0;
140    data_location = 0;
141
142
143    // Data 45 at data_cache[0]
144    #2
145    instruction = 0;
146    instruction_memory_cycle = 1'b0;

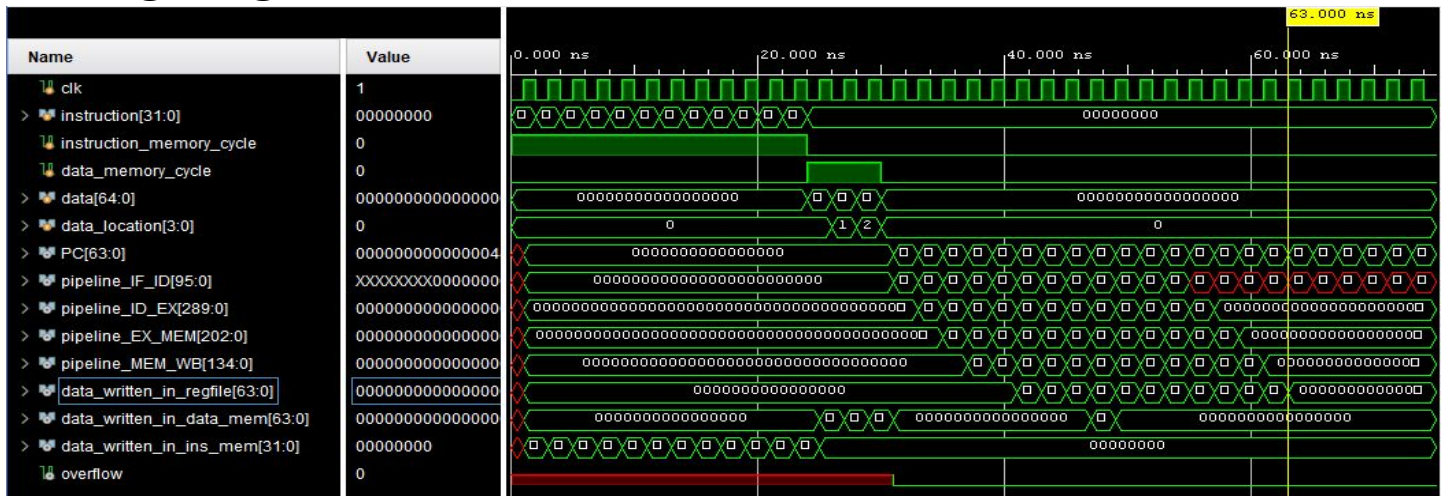
```

```

147 ○ data_memory_cycle = 1'b1;
148 ○ data = 45;
149 ○ data_location = 0;
150
151 // Data -20 at data_cache[1]
152 ○ #2
153 ○ instruction = 0;
154 ○ instruction_memory_cycle = 1'b0;
155 ○ data_memory_cycle = 1'b1;
156 ○ data = 20;
157 ○ data_location = 1;
158
159 // Data 20 at data_cache[2]
160 ○ #2
161 ○ instruction = 0;
162 ○ instruction_memory_cycle = 1'b0;
163 ○ data_memory_cycle = 1'b1;
164 ○ data = -20;
165 ○ data_location = 2;
166
167 #2
168 ○ instruction = 0;
169 ○ instruction_memory_cycle = 1'b0;
170 ○ data_memory_cycle = 1'b0;
171 ○ data = 0;
172 ○ data_location = 0;
173 end
174
175 ○ always #1 clk = ~clk;
176
177 ○ initial #75 $finish;
178 endmodule

```

Timing Diagram:



Description of the timing diagram for written test bench:

The register values in each cycle can be observed from the above timing diagram. **pipeline_IF_ID**, **pipeline_ID_EX**, **pipeline_EX_MEM**, **pipeline_MEM_WB**, represents the pipeline register values. **data_written_in_regfile** represents the data written back in the register file. **data_written_in_data_mem** represents the data written in the data cache. **data_written_in_ins_mem** represents the data written in the ins cache.

Initially 12 instructions are written in ins-cache including one stall. Data values 45, 20, -20 are written at the locations 0, 1, 2 of data cache. We can observe that in total 5 clock cycles are taken to completely run a instruction, ld – the critical instruction requires all the 5-stages completely.

□ Analysis of timing diagram with the help of test bench written:

After writing the instructions in the instruction cache and data values in data cache the scenario is:

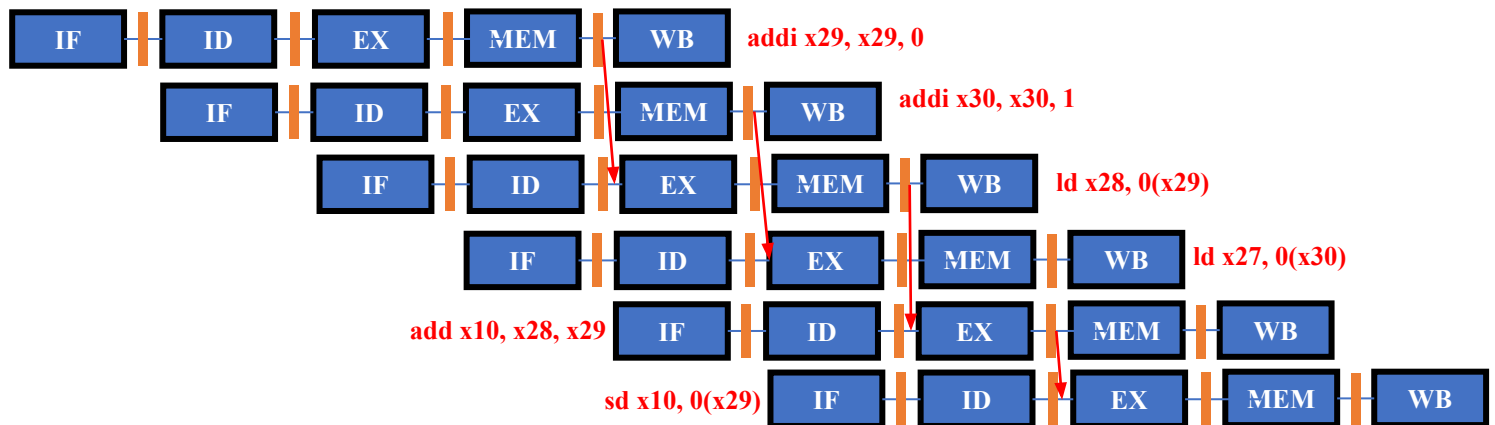
Instruction Cache:

Address	Instrucion	Description
0000	addi x29, x29, 0	# $x29 = 0 + 0 = 0$
0001	addi x30, x30, 1	# $x30 = 0 + 1 = 1$
0010	ld x28, 0(x29)	# $x28 = \text{MEM}[0 + 0] = 45$
0011	ld x27, 0(x30)	# $x27 = \text{MEM}[0 + 1] = 20$
0100	add x10, x28, x29	# $x10 = 45 + 0 = 45$
0101	sd x10, 0(x29)	# $\text{MEM}[0 + 0] = 45$
0110	ld x11 0(x29)	# $x11 = \text{MEM}[0 + 0] = 45$
0111	add x0, x0, x0	# [STALL]
1000	mul x6, x11, x31	# $x6 = 45 * 1 = 45$
1001	ld x18, 1(x30)	# $x18 = \text{MEM}[1 + 1] = -20$
1010	add x0, x0, x0	# [STALL]
1011	sub x19, x27, x18	# $x19 = 20 - (-20) = 40$

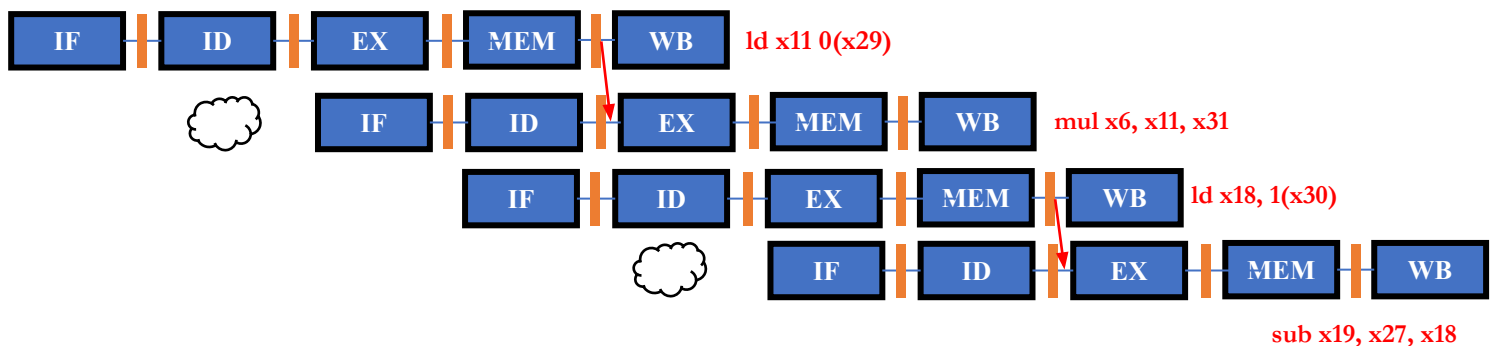
Data-cache:

Address	Data
0000	45
0001	20
0010	-20

The forwarding lines are indicated in red. The below picture depicts the execution of first 6 instructions. As there is full forwarding there is no requirement of STALLs in first 6 instructions.



In the below picture instructions and its computation is shown along with full forwarding:



Including 2 stalls and 10 instructions on total we can say that 12 instructions are executed. Here stalls are implemented by `add x0, x0, x0`.

Total Number of cycles for this 10 instructions(2-stalls) = 16

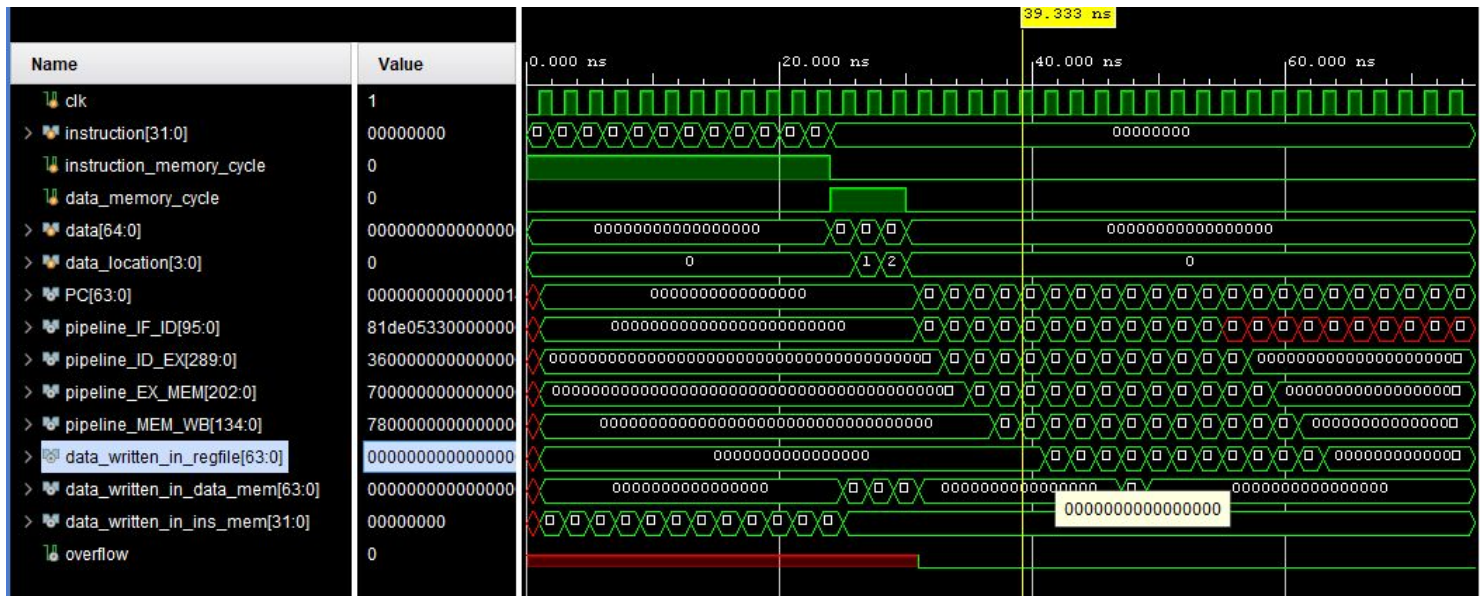
Actual Number of instructions = 10

Stalls = 2

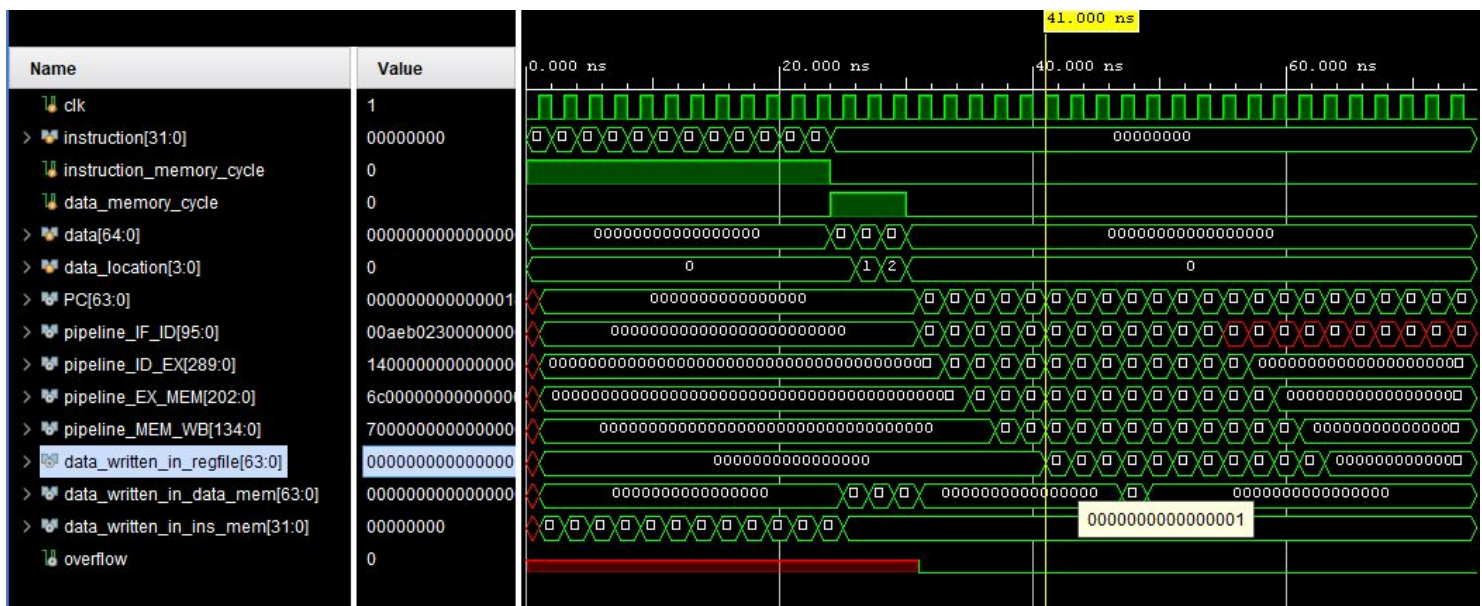
[average CPI]: $CPI = (16) / (10) = 1.6$ (2-stalls are not considered as instructions)

The CPI is not 1(ideal) because some cycles are wasted in stalling and also time required to get full pipelining.

Instruction: **addi x29, x29, 0.** so `data_written_in_regfile = 0` :



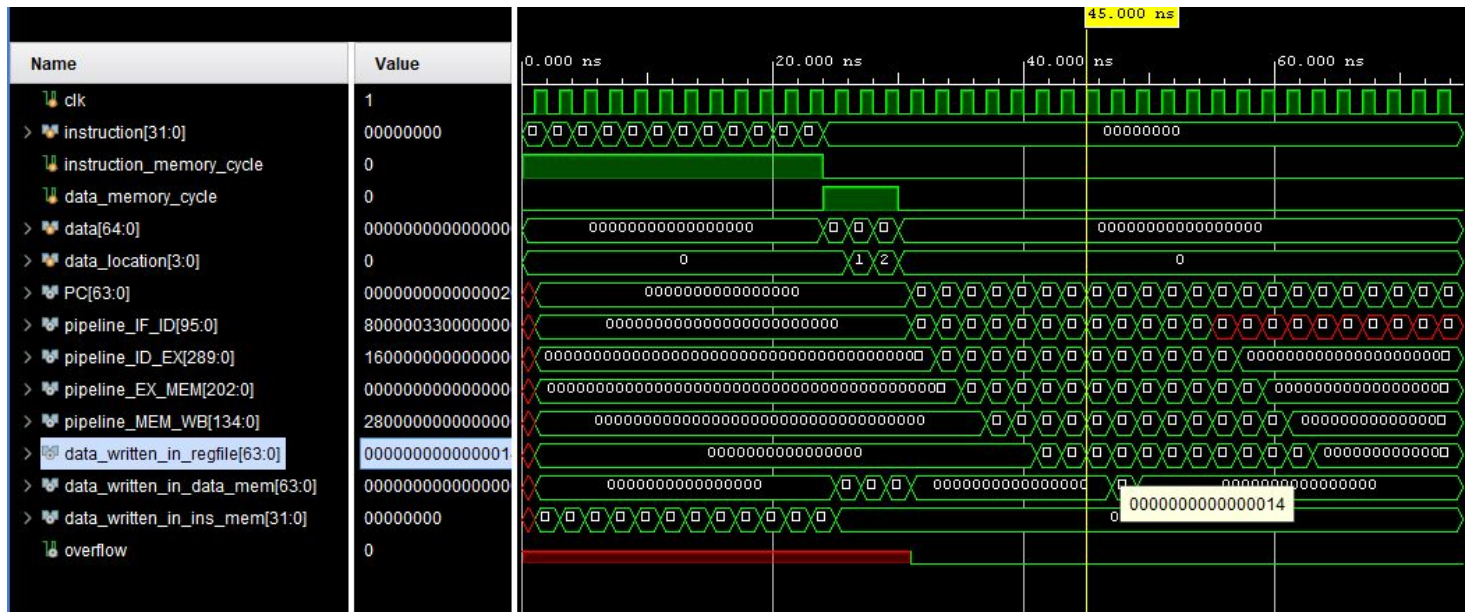
Instruction : **addi x30, x30, 1.** so `data_written_in_regfile = 1`.



Instruction : **ld x28, 0(x29).** so `data_written_in_regfile = MEM[0] = 45` (2d in hexadecimal)



Instruction: **ld x27, 0(x30)**. so $\text{data_written_in_regfile} = \text{MEM}[1] = 20$ (14 in dexadecimal) :



Instruction : **add x10, x28, x29**. so $\text{data_written_in_regfile} = 45 + 0 = 45$. (2d in hexadecimal)



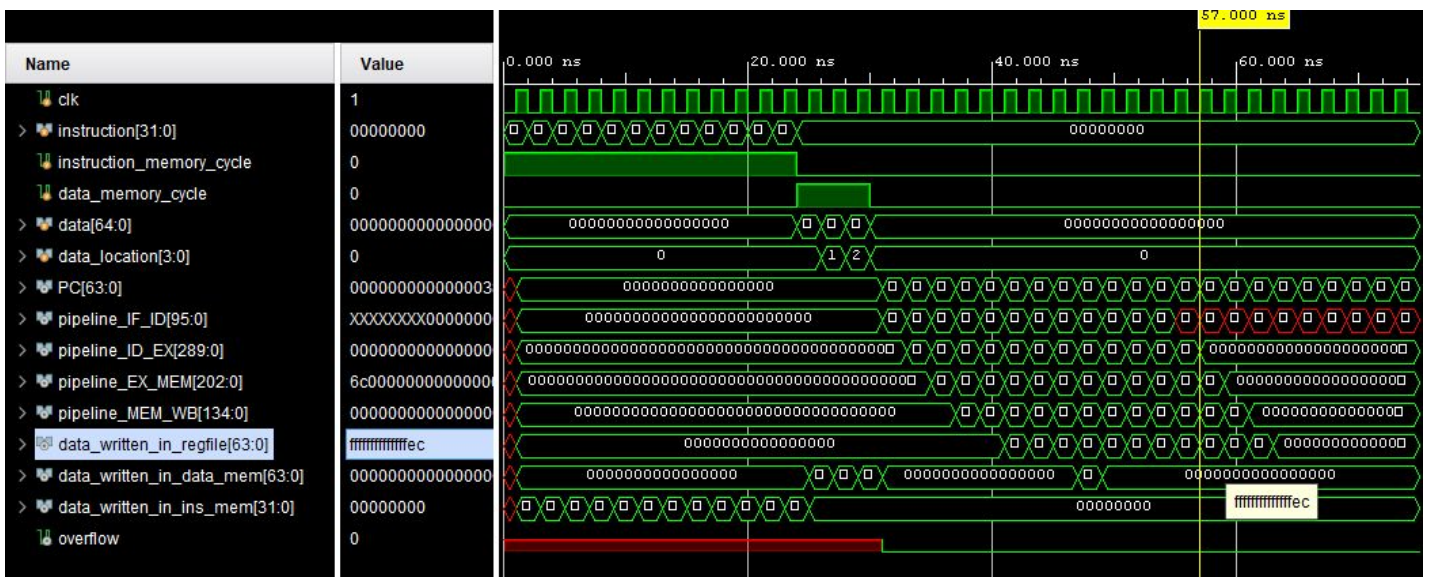
Instruction : **sd x10, 0(x29)**. so $\text{data_written_in_datamem} = 45$ (2d in hexadecimal):



Instruction : **mul x6, x11, x31**. Data written in reg file = $45 * 1 = 45$ [2d in hexadecimal]



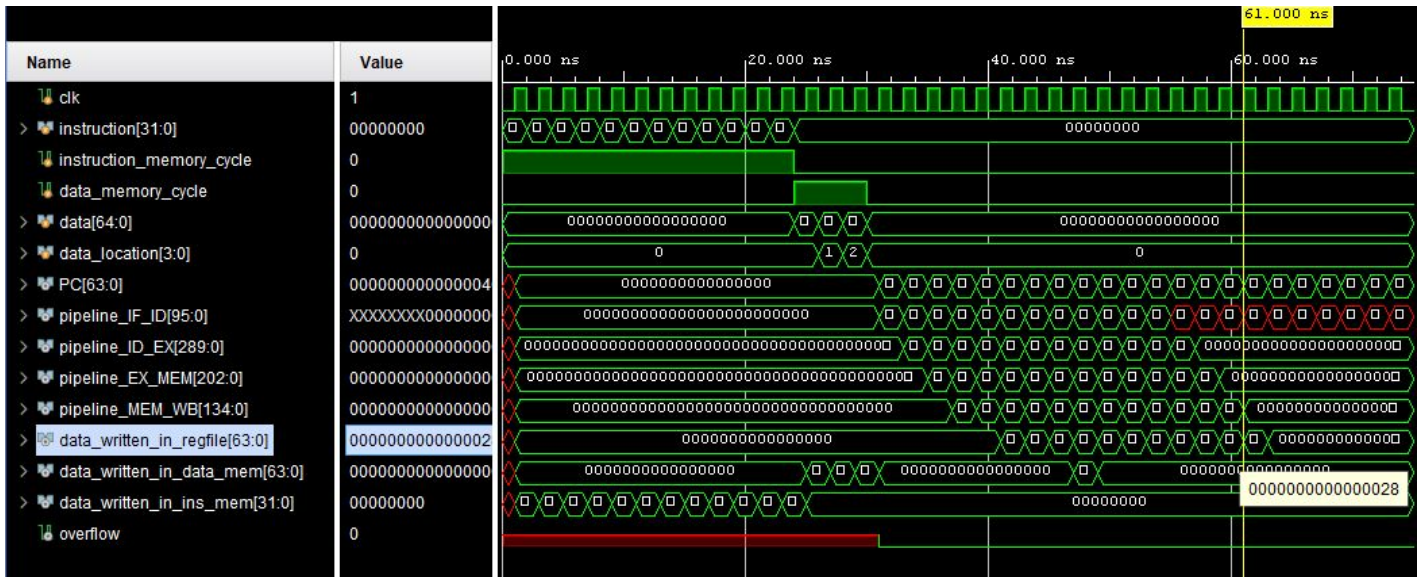
Instruction : **ld x18, 1(x30)**. so $\text{data_written_in_regfile} = \text{MEM}[2] = -20$:



Instruction : **add x0, x0, x0**. stall so $\text{data_written_in_regfile} = 0$:



Instruction : **sub x19, x27, x18**. Data written in reg file = $20 - (-20) = 40$ [28 in hexadecimal]



All the test cases are so far are passed successfully. If any further modifications are required it can be done by minor changes in Verilog code.

Time delays of written Verilog code:

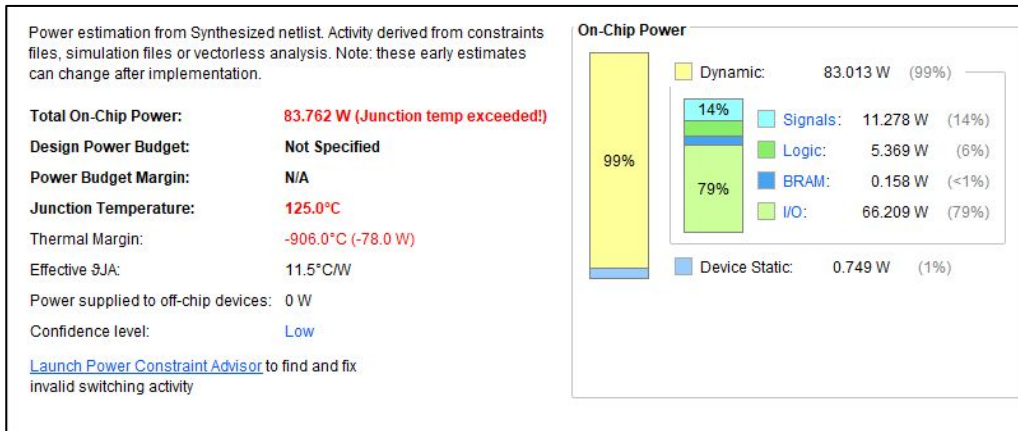
Unconstrained Paths - NONE - NONE												
Group Name: (none)												
From Clock:												
To Clock:												
Statistics												
Type						Total Endpoints						
Max Delay						5377						
Min Delay						5377						

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	143	143	161	pipeline_ID_EX_reg[216]/C	alu_result_reg[63]/D	120.448	56.604	63.844	∞				0.000
Path 2	∞	143	143	161	pipeline_ID_EX_reg[216]/C	pipeline_EX_MEM_reg[133]/D	120.448	56.604	63.844	∞				0.000
Path 3	∞	143	143	161	pipeline_ID_EX_reg[216]/C	alu_result_reg[62]/D	119.776	56.519	63.257	∞				0.000
Path 4	∞	143	143	161	pipeline_ID_EX_reg[216]/C	pipeline_EX_MEM_reg[132]/D	119.776	56.519	63.257	∞				0.000
Path 5	∞	141	141	161	pipeline_ID_EX_reg[216]/C	alu_result_reg[61]/D	117.809	55.673	62.136	∞				0.000
Path 6	∞	141	141	161	pipeline_ID_EX_reg[216]/C	pipeline_EX_MEM_reg[131]/D	117.809	55.673	62.136	∞				0.000
Path 7	∞	140	140	161	pipeline_ID_EX_reg[216]/C	alu_result_reg[60]/D	116.716	54.955	61.761	∞				0.000
Path 8	∞	140	140	161	pipeline_ID_EX_reg[216]/C	pipeline_EX_MEM_reg[130]/D	116.716	54.955	61.761	∞				0.000
Path 9	∞	137	137	161	pipeline_ID_EX_reg[216]/C	alu_result_reg[59]/D	114.012	53.970	60.042	∞				0.000
Path 10	∞	137	137	161	pipeline_ID_EX_reg[216]/C	pipeline_EX_MEM_reg[129]/D	114.012	53.970	60.042	∞				0.000

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 11	∞	1	1	2	pipeline_ID_EX_reg[0]/C	pipeline_EX_MEM_reg[0]/D	0.286	0.141	0.145	-∞				0.000
Path 12	∞	1	1	2	pipeline_ID_EX_reg[285]/C	pipeline_EX_MEM_reg[198]/D	0.286	0.141	0.145	-∞				0.000
Path 13	∞	1	1	2	pipeline_ID_EX_reg[286]/C	pipeline_EX_MEM_reg[199]/D	0.286	0.141	0.145	-∞				0.000
Path 14	∞	1	1	2	pipeline_ID_EX_reg[287]/C	pipeline_EX_MEM_reg[200]/D	0.286	0.141	0.145	-∞				0.000
Path 15	∞	1	1	2	pipeline_ID_EX_reg[288]/C	pipeline_EX_MEM_reg[201]/D	0.286	0.141	0.145	-∞				0.000
Path 16	∞	1	1	2	pipeline_ID_EX_reg[289]/C	pipeline_EX_MEM_reg[202]/D	0.286	0.141	0.145	-∞				0.000
Path 17	∞	1	1	2	pipeline_ID_EX_reg[3]/C	pipeline_EX_MEM_reg[3]/D	0.286	0.141	0.145	-∞				0.000
Path 18	∞	1	1	2	pipeline_IF_ID_reg[1]/C	pipeline_ID_EX_reg[20]/D	0.286	0.141	0.145	-∞				0.000
Path 19	∞	1	1	2	pipeline_IF_ID_reg[2]/C	pipeline_ID_EX_reg[21]/D	0.286	0.141	0.145	-∞				0.000
Path 20	∞	1	1	2	pipeline_IF_ID_reg[3]/C	pipeline_ID_EX_reg[22]/D	0.286	0.141	0.145	-∞				0.000

Total critical path delay: 120.448 units.

Power consumption of written Verilog code:



Maximum Power Consumption is 83.762W.

- **Further extensions to make more realistic data path:**
 - 1) Adding additional hardware at ID stage for branch instructions to reduce the number of stalls for control Hazards (or) maintaining a branch predictor static or dynamic branch predictor.
 - 2) Increasing the number of stages so that on an average for many instructions the average performance can be increased significantly.